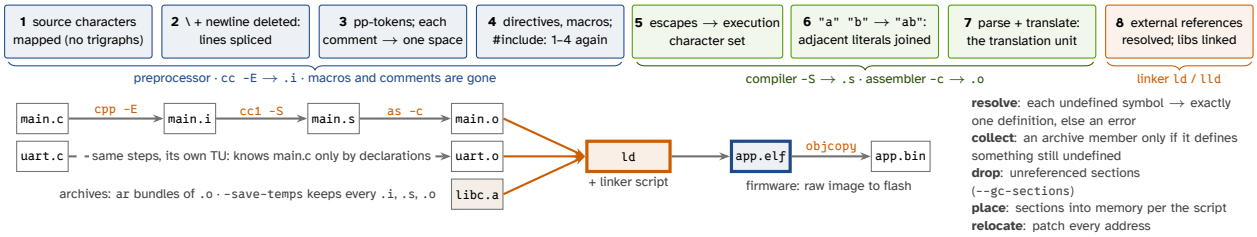
Undefined behaviour & the C build — preprocess, compile, link

In one line: Undefined behaviour is behaviour "for which this document imposes no requirements" (C23 3.5.3). The optimiser **assumes it never happens**, so checks vanish and loops change shape — often only at -O2, far from the bug. **Building** = per *translation unit* preprocess → compile → assemble to a `.o`; then the **linker** resolves every symbol to one definition, places the sections and patches the addresses.

Translation phases (C23 5.1.1.2) — conceptual; real tools fold them together



Four kinds of behaviour (C23 3.5)

kind	the standard	its own example
undefined	**no requirements**; J.2 lists 221	null pointer dereference
unspecified	≥2 options, choice not documented	order of evaluating arguments
impl.-defined	unspecified, choice documented	>> of a negative value
locale-specific	per locale, documented	`islower` beyond a–z

A *constraint* violation needs a diagnostic even if it is also UB (5.1.1.3); UB alone needs none. C23 note 3: observable behaviour *before* the UB still happens as specified.

The catalogue

UB (clause)	example	defined instead
signed overflow 6.5.1	`INT_MAX + 1`, `INT_MIN / -1`	unsigned wraps; `ckd_add`
shift 6.5.8	count <0 or ≥ width; `1 << 31`; `-1 << 1`	`1u << 31`
divide by 0 6.5.6	`x / 0`, `x % 0`	float: ±∞, NaN (Annex F)
pointer maths 6.5.7	past one-past-the-end, even unread; `p - q`, `p < q` across objects	== across objects
lifetime 6.2.4	use after `free`; a returned `&local`	`p = NULL` after `free`
uninitialised 6.3.2.1	`int x; return x;` (no &x)	`int x = {};`
null 3.5.3	`p->f` with `p == NULL`	test before first use
effective type 6.5.1	`*(uint32_t *)&f`; `char` may alias all	`memcpy`; a union
misaligned 6.3.2.3	`(uint32_t *)(buf + 1)`: the cast	`memcpy` to a local
unsequenced 6.5.1	`i = ++i + 1; a[i++] = i;`	one write per statement
data race 5.1.2.5	2 threads, ≥1 write, no sync	`_Atomic`, mutex
string literal 6.4.5	`char *s = "hi"; s[0] = 'H';`	`char s[] = "hi";`
2 declarations 6.2.7	`extern long n;` vs `int n;`	one header for both
fn pointer 6.3.2.3	call through another function type	a wrapper
no `return` 6.9.2	caller uses the missing value	only `main` gets 0

Not UB, same effect: a loop whose condition is not constant and that does no I/O, `volatile` or atomic access may be *assumed to terminate* (6.8.6.1) — empty ones are deleted; `while (1)` is not.

What the optimiser does with it

```
int dev_read(struct dev *d) {
  int v = d->val; // *d ⇒ d != NULL
  if (!d) return -1; // so: dead
  return v; }
```

-O2 →

```
int dev_read(struct dev *d) {
  return d->val; }
// the null test is gone
```

- Overflow "cannot happen" ⇒ `x + 1 > x` → `true`, `x * 2 / 2` → `x`, `for (i = 0; i <= n; i++)` runs exactly $n+1$ times. Type rules ⇒ a `float` store cannot change a `float *`: a store loop becomes one `memset` (Lattner, 2011).
- Linux 2.6.30 `tun` (CVE-2009-1897): `tun->sk` read before `if (!tun)`; GCC deleted the test → local privilege gain.
- Fine at -O0, broken at -O2 or with the next compiler = latent UB. Fix the code; a lower -O only hides it.

GCC flag	switches the assumption off
`-ftrapv`	trap on signed overflow
`-fwrapv` · `-fwrapv-pointer`	signed + − × · pointer + − wrap
`-fno-strict-overflow`	both of the above
`-fno-strict-aliasing`	no type-based aliasing (on at -O2 -O3 -Os)
`-fno-delete-null-pointer-checks`	keep null tests (deleted by default on most targets)

Linux builds with the last three: a GCC *dialect*, not C.

C23: new tools, same rules

Two's complement is now the only representation — signed overflow is **still** UB. `<stdckdint.h>`: `ckd_add(&r, a, b)` (`_sub`, `_mul`) computes as if infinitely wide, stores the wrapped result, returns `true` on overflow. New: `unreachable()` (reaching it is UB), `nullptr`, `typeof`, `memset_explicit`, `[[nodiscard]]`, `[[fallthrough]]`, `[[maybe_unused]]`. Newly UB: `realloc(p, 0)`. Gone: K&R definitions; `int f();` means `f(void)`. GCC 15+ defaults to `-std=gnu23` (Clang 23 still `gnu17`); GCC 14 turned implicit declarations, implicit `int`, int-conversion and incompatible pointers into errors (`-fpermissive` undoes it).

Catch it

`-Wall -Wextra` is not "all": add `-Wconversion -Wshadow -Wundef -Wcast-align -Wvla -Wformat=2 -Wnull-dereference` (the last only when optimising). `-Wall` brings `-Wstrict-aliasing=3` and (GCC 15) `-Wheader-guard`. Static analysis: `gcc -fanalyzer` (GCC 10+), `clang --analyze`, `scan-build make`.

`-fsanitize=`	finds — only on paths a run executes	cost · limits
`address`	out of bounds (heap, stack, globals), use after free / return / scope, double free; leaks (Linux)	~2×; not with `thread`
`undefined`	signed overflow, shift, ÷0, null, misalignment, bounds — not unsigned wrap	reports, continues; `-fsanitize-trap`
`thread`	data races	5–15× CPU
`memory`	reads of uninitialised memory	Clang, 3×; instrument all

Ship hardened (OpenSSF, Aug 2026): `-D_FORTIFY_SOURCE=3` (-O1+), `-fstack-protector-strong`, `-fstack-clash-protection`, `-ftrivial-auto-var-init=zero`, `-Wl,-z,relro,-z,now`, the three `-fno-…` above; GCC 14+: `-fhardened`, most in one flag.

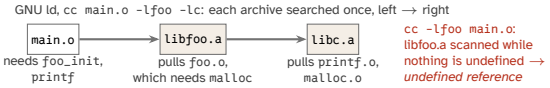
Optimisation levels

-O0	default: fastest compile, most debuggable · -Og: GCC's "edit-compile-debug" level, ESP-IDF's default; Clang: like -O1
-O2	GCC: all that does not trade size for speed; strict aliasing; loop vectorising
-O3	+ slower or larger passes · -Os, -Oz: for size, harder for size
-Ofast	-O3 -ffast-math — not IEEE; deprecated in Clang 19

Linkage, archives, shared libraries

External (file-scope default): one entity across all TUs. **Internal**: `static` at file scope. **None**: locals (`static` there = storage). A used external name needs **exactly one** external definition (6.9.1; C++: the ODR): two → *multiple definition*, none → *undefined reference*.

- Header *declares* `extern int count;`, one `.c` *defines* it. `int count;` in a header is a **tentative definition** per TU — merged as "common" until GCC 10 / Clang 11 made `-fno-common` the default.
- C `inline` with no `extern` declaration in the TU is only an inline definition: an un-inlined call (-O0) needs an external one elsewhere. Headers: `static inline`.
- `__attribute__((weak))`: a default any normal definition overrides. LTO (`-flto`, `=thin`): objects carry IR, the link optimises across TUs.



- Users before providers; cycles need `--start-group … --end-group` ("significant performance cost"). lld remembers every archive seen; `--warn-backrefs` flags lines GNU ld would reject.
- `.so`/`.dylib`/`.dll`: mapped by the dynamic loader at run time, shared, replaceable; `-fPIC`; `-fvisibility=hidden` exports only what you mark.
- ESP-IDF builds with `-ffunction-sections -fdata-sections -Wl,--gc-sections`. In ROM: `.data` stored in flash (LMA), copied to RAM (VMA) at start-up. `-Wl,-Map=app.map`, `idf.py size-components`: who uses the flash.

Headers & macros

```
#ifndef UART_H                  /* guard: the whole file */
#define UART_H
#define SQ(x) ((x) * (x))       // parenthesise args AND body
#define SWAP(a, b) do { int t_ = (a); (a) = (b); (b) = t_; } while (0)
static inline int sq(int x) { return x * x; }  // typed, x once
#endif
```

`SQ(i++)`: two unsequenced writes, UB. `do{}while(0)` lets `if (c) SWAP(x, y); else` parse. `#if FOO`, FOO undefined: silently 0 (`-Wundef`). `#pragma once` is not in every preprocessor.

Remember: UB is a promise to the optimiser; -O0 only hides a broken one.

Related: `c-memory-layout` · `c-pointers-const-restrict` · `c-volatile-atomics-isr` · Sanitizers & runtime checkers · Release-only bugs · Cross-language FFI